Reto práctico: API Explorer

Desarrollo de Aplicaciones Móviles | Ionic + Angular Standalone + Capacitor

Modalidad: individual o parejas • Taller presencial: 2 horas • Continuación autónoma: preparación para la prueba de la próxima semana

1. Situación y objetivo

Una organización necesita una aplicación móvil que explore un catálogo de información real proporcionado por una API pública (libros, países, videojuegos, recetas, películas, etc.). El reto consiste en integrar una API desconocida, implementar navegación, búsqueda y carga incremental, y organizar el código mediante servicios y modelos. La meta de clase es avanzar lo máximo posible; la finalización en casa servirá como repaso práctico para la evaluación.

2. Elección de la API

Elegir una API distinta de PokéAPI desde https://github.com/public-apis/public-apis o https://free-apis.github.io/#/. Antes de programar, verificar disponibilidad, condiciones de uso, CORS y capacidad de paginación (page/limit, offset/limit, cursor o equivalente). Preferir APIs sin clave secreta. Si la API no soporta paginación, justificar la elección y acordar con el docente una alternativa; no simular infinite scrolling cargando repetidamente los mismos datos.

Ejemplos orientativos: Open Library (búsqueda de libros), RAWG (videojuegos, sujeto a clave y condiciones), TheMealDB (recetas), TMDB (películas, sujeto a autenticación). Verificar endpoints y límites en la documentación oficial antes de elegir.

3. Desafío funcional obligatorio

Código	Funcionalidad	Criterio observable
R1	Listado remoto	Tarjetas con información real y al menos 2 atributos por elemento.
R2	Infinite scrolling	ion-infinite-scroll: solicitar página siguiente, anexar sin duplicar y detener al agotarse.
R3	Búsqueda y filtros	ion-searchbar; reiniciar resultados y paginación al cambiar la consulta.
R4	Detalle y rutas	Vista de detalle por identificador o parámetro de ruta; regreso funcional.
R5	Estados y errores	Loading, lista vacía, error de red y reintento sin bloquear la interfaz.
R6	Favoritos persistentes	Agregar/quitar favoritos con Capacitor Preferences o equivalente.
R7	Estructura de código	Servicio HTTP, interfaces TypeScript y componentes/páginas

	separados.

Condiciones técnicas del infinite scrolling

- Utilizar ion-infinite-scroll e ion-infinite-scroll-content, no un botón “cargar más” como sustituto.
- Mantener estado de página/cursor, loading, hasMore y lista acumulada.
- Evitar solicitudes simultáneas; llamar complete() al terminar, incluso ante error.
- Deshabilitar el infinite scroll cuando no existan más registros.
- Al iniciar otra búsqueda, limpiar resultados, restablecer paginación y evitar mezclar respuestas antiguas con nuevas.

4. Plan de trabajo presencial — 120 minutos

Tiempo	Actividad	Meta verificable
0–15 min	Elegir API, revisar JSON y paginación.	Endpoint probado y datos identificados.
15–35 min	Crear proyecto Ionic Standalone, páginas, rutas y servicio.	App arranca; servicio y modelo creados.
35–65 min	Conectar API y construir tarjetas/listado.	Primera página visible con datos reales.
65–95 min	Integrar ion-infinite-scroll y control de estado.	Se cargan 2 o más páginas sin duplicados.
95–110 min	Incorporar búsqueda y reinicio de paginación.	Nueva búsqueda muestra resultados correctos.
110–120 min	Commit, README parcial y demostración de avance.	Repositorio actualizado con evidencia.

La meta prioritaria de las dos horas es R1 + R2 + estructura R7. No se exige terminar todos los requisitos durante la sesión.

5. Trabajo autónomo: estudiar para la prueba

Completar R3–R6 y refactorizar lo realizado en clase. Prepararse para explicar y reproducir sin copiar: creación de proyecto/páginas/servicios, HttpClient, interfaces, async/await u Observables, rutas, binding, eventos, condicionales, listas, estados de carga y ciclo de infinite scrolling.

Preguntas de autoevaluación

- ¿Por qué conviene consumir la API desde un servicio en vez de directamente desde cada página?
- ¿Qué cambia entre reemplazar un arreglo y concatenar nuevos resultados?
- ¿Cómo evitar duplicados y solicitudes simultáneas al llegar al final del listado?
- ¿Qué se debe reiniciar cuando cambia el texto de búsqueda?
- ¿Qué ocurre si falla la tercera página? ¿Cómo recuperar la interfaz?
- ¿Cómo diferenciar “sin resultados” de “error de conexión”?
- ¿Qué información se almacena localmente y qué se obtiene de la API?

6. Entregables y evidencias

- Al finalizar la clase: URL del repositorio GitHub, commit del avance y captura o video corto mostrando el listado y, si se alcanzó, infinite scrolling.

- Antes de la prueba (fecha que indicará el docente): repositorio actualizado, README con API/endpoints, instalación, decisiones de paginación, capturas y dificultades; APK de Android o evidencia de ejecución en dispositivo/emulador.
- Demostración individual breve: cada integrante debe poder explicar el servicio, la búsqueda, la paginación y el manejo de errores.

7. Rúbrica orientativa (100 puntos)

Criterio	Puntos
Consumo real de API y modelos/servicio	20
Infinite scrolling robusto (paginación, estados, fin)	25
Búsqueda y reinicio correcto de resultados	15
Navegación y pantalla de detalle	10
Favoritos y persistencia local	10
Errores, carga y estados vacíos	10
GitHub, README y explicación técnica	10
TOTAL	100

Nota: El avance presencial se revisará de manera formativa; la rúbrica corresponde al producto completo. Se valorará el trabajo propio y la capacidad de explicar el código, especialmente de cara a la prueba.

8. Extensión opcional

Para quienes terminen antes: implementar pull-to-refresh con ion-refresher, caché local o filtros combinados con búsqueda y paginación. Estas extensiones no reemplazan los requisitos obligatorios.